DATA ENGINEERING REPORT

COURSE INSTRUCTOR: DR. CHANDRA SEKAR V

NAME: SHARVAANI NAGUBANDI

ROLL NO: 23BCS230

A1

4e8ab61 (HEAD -> main, origin/main) Restore correct train test split

ced0a9a Test CI failure with incorrect split ratio

aec6f37 Fix Python test execution in CI

249acfc Add Python package initialization files

a751c2c Add Iris data preparation test and CI workflow

bff8746 Add required project folders

Screenshots to include in the appendix

A2

Command	Where the change happens	What exactly happens?
git init	Working folder / Local git.	Initializes a new Git repository in the current folder and creates a hidden .git directory.
git add train.py	Staging area	Adds changes made in train.py to the staging area, preparing them for commit.
git commit -m "msg"	Local Git repository	Saves the staged changes as a new commit in the local repository with the given message.
git remote add origin URL	Local Git configuration	Connects the local repository to a remote GitHub repository by assigning it the name origin

git push -u origin main	Remote GitHub repositoryUploads local commits from the main branch to GitHub and sets the upstream tracking branch.	Uploads local commits from the main branch to GitHub and sets the upstream tracking branch.
git pull origin main	Local Git repository and working folder	Downloads new commits from the remote main branch and merges them into the current local branch.

No. After running git commit, the changes are saved only in the local Git repository on our computer. They are not automatically uploaded to GitHub. To make the changes visible to teammates on GitHub, we must execute git push

A3. A Small CI Workflow

A Pytest test was created in tests/test_prepare_data.py to verify that the Mage prepare_data block correctly splits the Iris dataset into 80% training data (120 records) and 20% testing data (30 records).

A GitHub Actions workflow was configured in .github/workflows/ci.yml to run automatically on every push. It checks out the repository, sets up Python 3.12, installs the required packages from requirements.txt, and executes the Pytest test.

To verify the workflow, the test_size parameter was temporarily changed to 0.3, resulting in a failing CI run. The original value was then restored, and the correction was committed and pushed. The subsequent GitHub Actions run passed successfully.

Screenshots included in the appendix

B1 Calculation of rejected percentage:

\[\frac{2}{8}\times100=25\% \]

The FreshCart ELT pipeline was successfully executed using Snowflake. The pipeline extracted 8 raw order records: 6 from the web source and 2 from the mobile source.

During the cleaning stage, 6 records passed validation, while 2 records were moved to quarantine. ORD-1004 was rejected because of a non-positive amount. ORD-1006 was rejected because of an invalid email address and pincode.

After deduplication, 4 unique orders remained. These records were processed into the Gold layer.

The final Gold summary showed:

- Total orders: 4
- Total revenue: ₹7,785.00

- Total loyalty points: 126

The rejected share was 25%, calculated as 2 rejected records out of 8 raw records.

Thus, the baseline pipeline executed successfully and produced the expected results

Screenshot in the appendix

B2

1. Raw records in BRONZE.ORDERS_RAW and differences in field names

ORD-1003 appears twice in the Bronze table because it was received from two different sources: the website and the mobile application.

Website record (INGEST_SEQ = 2):

- Source: web
- order_id: ORD-1003
- amount: NULL
- currency: INR
- cust_email: meera@outlook.com
- cust_name: MEERA S
- updated_at: 2026-09-02 09:10:00

Mobile application record (INGEST_SEQ = 8):

- Source: mobile
- orderId: ORD-1003
- total.value: 2,100.00
- total.currency: INR
- customer.email: Meera@Outlook.com
- customer.name: Meera S
- updatedAt: 2026-09-02T18:40:00

Why are the field names different?

The website and mobile application use different data formats (schemas). The website stores fields such as order_id, amount, and cust_email directly, whereas the mobile application uses fields such as orderId, total.value, and customer.email. Therefore, the ELT pipeline must standardize these different formats into common columns before further processing.

The mobile record also contains the actual order amount of ₹2,100.00 and has a later update timestamp than the website record.

2. SQL expressions that convert both records into the same columns in SILVER.ORDERS_CHECKED

The SQL file 10_silver_orders_clean.sql uses a CTE called unified to combine website and mobile records using UNION ALL.

It maps the differently named fields into common columns:

Common column	Website expression	Mobile expression
order_id	raw:order_id::STRING	raw:orderId::STRING
cust_email	raw:cust_email::STRING	raw:customer.email::STRING
cust_name	raw:cust_name::STRING	raw:customer.name::STRING
order_date	raw:order_date::STRING	raw:orderDate::STRING
amount	raw:amount::STRING	raw:total.value::STRING
currency	raw:currency::STRING	raw:total.currency::STRING
pincode	raw:pincode::STRING	raw:deliveryPin::STRING
updated_at	raw:updated_at::STRING	raw:updatedAt::STRING

After combining the records, the typed CTE performs standardization:

- TRIM(order_id) removes extra spaces.
- LOWER(TRIM(cust_email)) converts email addresses to lowercase and removes spaces.
- INITCAP(RTRIM(TRIM(cust_name), '.')) standardizes customer names and removes trailing full stops.
- COALESCE() with TRY_TO_DATE() handles different date formats.
- TRY_TO_DECIMAL(REGEXP_REPLACE(amount, '[^0-9.-]', ''), 12, 2) removes currency symbols and converts the amount into a decimal value.
- UPPER(TRIM(currency)) standardizes currency codes.
- TRY_TO_TIMESTAMP_NTZ(updated_at) converts update timestamps into a common timestamp format.

Thus, both website and mobile records are converted into a common structure in SILVER.ORDERS_CHECKED, making further cleaning and deduplication possible.

Question 3: Which copy survives deduplication and why?

The mobile copy of ORD-1003 survives in silver.orders_dedup.

The SQL uses PARTITION BY order_id to group records belonging to the same order. It then uses ORDER BY updated_at DESC to rank the newest updated record first. If two records have the same update time, ingest_seq DESC gives priority to the record that arrived later. Finally, ROW_NUMBER() = 1 keeps only the first record from each group.

For ORD-1003, the web record was updated at 09:10, whereas the mobile record was updated at 18:40 on 2 September 2026. Therefore, the mobile record survives and the web record is discarded.

Question 4: How are amount_inr and loyalty_points calculated in Gold?

The Gold table calculates amount_inr by multiplying the original order amount by the exchange rate for that order's currency and date. The result is rounded to two decimal places.

Formula:

amount_inr = ROUND(amount × fx_rate, 2)

Loyalty points are calculated using:

loyalty_points = FLOOR(amount_inr / 100) × tier_multiplier

Calculations:

- **ORD-1001:** ₹1,250 × 1.0000 = ₹1,250. Loyalty points = FLOOR(1250/100) × 1 = 12 points.
- **ORD-1003:** ₹2,100 × 1.0000 = ₹2,100. Loyalty points = FLOOR(2100/100) × 3 = 63 points.
- **ORD-1005:** ₹899 × 1.0000 = ₹899. Loyalty points = FLOOR(899/100) × 2 = 16 points.

Why does Priya get different multipliers?

Priya's ORD-1001 is dated 1 September 2026, when her tier was Silver. Therefore, she gets a multiplier of 1 and earns 12 points.

Her ORD-1005 is dated 3 September 2026, when her tier was Gold. Therefore, she gets a multiplier of 2 and earns 16 points.

The table responsible for maintaining customer tier history is DIM.CUSTOMERS_SCD2. The join matches the customer's email and checks whether the order date falls between valid_from and valid_to. The REF.TIER_MULTIPLIER table then provides the multiplier corresponding to the selected tier.

B3. Adding Unsupported Currency Validation

A new validation rule was added to 10_silver_orders_clean.sql to reject currencies other than INR and USD.

The rule added was:

IFF(currency NOT IN ('INR', 'USD'), 'unsupported_currency', NULL)

After rerunning the pipeline, ORD-1006 had three validation errors: bad_email, bad_pincode, and unsupported_currency.

The Gold output remained unchanged, with 4 orders, ₹7,785 revenue and 126 loyalty points, because ORD-1006 was still quarantined.

The quarantine table contained two records for ORD-1006. The first record had the original two validation reasons, while the second contained all three reasons. This happened because the NOT EXISTS condition checks the source, order ID and update timestamp. It does not update the reasons in an existing quarantine record. Since the existing record was not updated, another record was inserted with the new reasons.

Thus, the new validation rule worked, but the quarantine logic allowed a duplicate record when the validation reasons changed.

B4. Customer Tier Change and Gold Table Update

1. SCD Type 2 history for Arjun (CUST-02)

After adding the Gold tier event with effective date 2026-08-15, the customer history contains two records:

Tier	Valid From	Valid To
Silver	2024-01-15	2026-08-14
Gold	2026-08-15	9999-12-31

The previous Silver record was closed one day before the new Gold record became effective. This preserves the customer's tier history.

2. Impact on ORD-1002

ORD-1002 was placed on 2026-09-01, so the pipeline uses Arjun's Gold tier.

- Previous tier: Silver
- New tier: Gold
- Previous loyalty points: 35
- New loyalty points: 70

3. Impact on Gold table

Before the change, total Gold loyalty points were 126. After the change, they increased to 161.

Only one Gold row, ORD-1002, received a new published_at timestamp. The other three orders retained their previous timestamps because their values did not change.

4. How row_hash and MERGE make the pipeline safe to rerun

The pipeline calculates a row_hash using the important order values, including customer tier, amount, FX rate and loyalty points. The MERGE operation matches records using order_id. If an existing order has a different row_hash, its values are updated and published_at is refreshed. If the hash is unchanged, the existing record is left untouched. If the order does not exist in Gold, a new record is inserted. This makes the Gold publishing process idempotent, meaning repeated runs with unchanged data do not create duplicate Gold records or unnecessarily update existing ones.

5. Why Bronze grows by 8 rows per run

Every pipeline run extracts 6 web orders and 2 mobile orders, adding 8 new records to BRONZE.ORDERS_RAW. Bronze stores incoming raw records as an append-only history, so it does not overwrite previous records. Deduplication happens later in the Silver layer, not in Bronze.

. B5. Additional Data Quality Check

1. New quality check added

I added a future-date validation check in run_pipeline.py.

SELECT COUNT_IF(order_date > CURRENT_DATE())

FROM silver.orders_dedup;

This check counts orders whose order date is later than the current date. If the count is zero, the check passes; otherwise, it fails.

2. Testing the quality check

I added a temporary mock order ORD-1099 with the date 2099-01-01 to test the validation rule.

During the test, the pipeline displayed:

orders with future order_date 1 FAIL

quality checks failed: orders with future order_date = 1

This confirmed that the new quality check correctly detected future-dated orders.

3. Removing the mock order

After testing, I removed ORD-1099 from mock_data.py and deleted its record from the Bronze and Gold tables. I reran the pipeline, and the future-date check returned 0 OK.

4. ELT vs ETL

For the FreshCart project, ELT is suitable because raw data from different sources can first be loaded into Snowflake and then transformed using SQL. It preserves the original raw data and allows different formats from web and mobile orders to be processed together. ELT also uses Snowflake's processing capabilities for scalable transformations. ETL may be preferable when sensitive data must be cleaned, masked or filtered before it is loaded into the data warehouse.

PASTING PROBLEM OUTPUT

/content/freshcart_elt 20:39:55 pipeline run_id=20260930_203955 run_date=2026-09-30 mock=True 20:39:55 extract web: 7 records -> /content/freshcart_elt/landing/web/2026-09-30/web__20260930_203955.json.gz 20:39:55 extract mobile: 2 records -> /content/freshcart_elt/landing/mobile/2026-09-30/mobile__20260930_203955.json.gz 20:39:55 extract crm: 5 records -> /content/freshcart_elt/landing/crm/2026-09-30/crm__20260930_203955.json.gz 20:39:55 extract fx: 1 records -> /content/freshcart_elt/landing/fx/2026-09-30/fx__20260930_203955.json.gz 20:39:56 snowflake.connector Python connector starting v5.2.0 20:39:56 snowflake.connector._common.extras pyarrow is installed (version: 23.0.1) 20:39:56 numexpr.utils NumExpr defaulting to 2 threads. 20:39:56 snowflake.connector._common.extras pandas is installed (version: 2.2.3) 20:39:56 snowflake.connector._common.extras numpy is installed (version: 2.4.6) 20:39:56 snowflake.connector._common.extras tzlocal is installed (version: 5.4.5) 20:39:57 snowflake.connector._common.extras sqlalchemy is installed (version: 2.1.7) /usr/local/lib/python3.13/dist-packages/snowflake/connector/connection/_connection.py:137: FutureWarning: enable_server_session_keep_alive_auto_detection was not set and defaults to True.

In a future version, the default will change to None. Please provide an explicit value: True = check for running async queries before logout (queries are preserved); False/None = always send logout on close (async queries may be terminated by server). Logout behavior can also be overridden with server_session_keep_alive. See the connection parameter docs for more info. options = self.config.to_proto_options(20:39:57 snowflake.connector._core outbound HTTP call method=PUT host=169.254.169.254 path=/latest/api/token 20:39:57 snowflake.connector._core HTTP response status=400 20:39:57 snowflake.connector._core failed to load region from IMDS err=failed to load IMDS session token: service error: invalid request parameters. This indicates an SDK bug. (FailedToLoadToken(FailedToLoadToken { source: ServiceError(ServiceError { source: TokenError { kind: InvalidParameters }, raw: Response { status: StatusCode(400), headers: Headers { headers: {"content-type": HeaderValue { _private: H1("text/plain; charset=utf-8") }, "metadata-flavor": HeaderValue { _private: H1("Google") }, "x-content-type-options": HeaderValue { _private: H1("nosniff") }, "date": HeaderValue { _private: H1("Wed, 11 Nov 2026 20:39:57 GMT") }, "content-length": HeaderValue { _private: H1("1") }} }, body: SdkBody { inner: Once(Some(b"\n")), retryable: true }, extensions: Extensions { extensions_02x: Extensions, extensions_1x: Extensions } } }) })) 20:39:57 snowflake.connector._core Proxy configuration: using environment variable proxy settings 20:39:57 snowflake.connector._core Starting Snowflake login process 20:39:57 snowflake.connector._core Making Snowflake login request login_url=https://TGDENIL-OW36864.snowflakecomputing.com/session/v1/login-request 20:39:57 snowflake.connector._core outbound HTTP call method=POST path=/session/v1/login-request attempt=1 20:39:57 snowflake.connector._core outbound HTTP call method=PUT host=169.254.169.254 path=/latest/api/token 20:39:57 snowflake.connector._core HTTP response status=400 20:39:57 snowflake.connector._core failed to load region from IMDS err=failed to load IMDS session token: service error: invalid request parameters. This indicates an SDK bug. (FailedToLoadToken(FailedToLoadToken { source: ServiceError(ServiceError { source: TokenError { kind: InvalidParameters }, raw: Response { status: StatusCode(400), headers: Headers { headers: {"content-type": HeaderValue { _private: H1("text/plain; charset=utf-8") }, "metadata-flavor": HeaderValue { _private: H1("Google") }, "x-content-type-options": HeaderValue { _private: H1("nosniff") }, "date": HeaderValue { _private: H1("Wed, 11 Nov 2026 20:39:57 GMT") }, "content-length": HeaderValue { _private: H1("1") }} }, body: SdkBody { inner: Once(Some(b"\n")), retryable: true }, extensions: Extensions { extensions_02x: Extensions, extensions_1x: Extensions } } }) })) 20:39:57 snowflake.connector._core outbound HTTP call method=PUT host=169.254.169.254 path=/latest/api/token 20:39:57 snowflake.connector._core HTTP response status=400 20:39:57 snowflake.connector._core provider failed to provide credentials provider=Ec2InstanceMetadata error=an error occurred while loading credentials: failed to load IMDS session token: service error: invalid request parameters. This indicates an SDK bug. (ProviderError(ProviderError { source: FailedToLoadToken(FailedToLoadToken { source: ServiceError(ServiceError { source: TokenError { kind: InvalidParameters }, raw: Response { status: StatusCode(400), headers: Headers { headers: {"content-type": HeaderValue { _private: H1("text/plain; charset=utf-8") }, "metadata-flavor": HeaderValue { _private: H1("Google") }, "x-content-type-options": HeaderValue { _private: H1("nosniff") }, "date": HeaderValue { _private: H1("Wed, 11 Nov 2026 20:39:57 GMT") }, "content-length": HeaderValue { _private: H1("1") }} }, body: SdkBody { inner: Once(Some(b"\n")), retryable: true }, extensions: Extensions { extensions_02x: Extensions, extensions_1x: Extensions } } }) }) })) 20:39:57 snowflake.connector._core Snowflake login completed successfully session_id=11872407565 session_validity_secs=3599 master_validity_secs=14399 session_params_count=52 server_version=10.40.103 20:39:57 pipeline ▶ setup 20:39:57 snowflake.connector._core Executing sync query request_id=2aac5e87-c631-4558-82cd-aff2ef7030fd 20:39:57 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:39:58

snowflake.connector._core Executing sync query request_id=4bfc50a3-fea2-4bbe-a2cd-161ef684d400 20:39:58 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:39:58 snowflake.connector._core Executing sync query request_id=de91edfb-7347-429f-870b-89f1bb60cb8f 20:39:58 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:39:58 snowflake.connector._core Executing sync query request_id=087e48d2-d16f-4bfd-af1c-b6e1c3c5ab4f 20:39:58 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:39:58 snowflake.connector._core Executing sync query request_id=9d6e8880-c538-489e-b2e5-cd25af3cb079 20:39:58 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:39:58 snowflake.connector._core Executing sync query request_id=0d7a7dde-b6bd-415b-86d2-7db530649b16 20:39:58 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:39:58 snowflake.connector._core Executing sync query request_id=a9d64145-39ec-41d6-b2c8-e9bf369918ce 20:39:58 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:39:58 snowflake.connector._core Executing sync query request_id=1ad89208-e20b-4cfd-a7a7-083327402d51 20:39:58 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:39:58 snowflake.connector._core Executing sync query request_id=6964bc4e-d059-4001-ab29-87cd8d3be431 20:39:58 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:39:58 snowflake.connector._core Executing sync query request_id=a93c1f27-5e05-48ec-8622-4ac82ba084cf 20:39:58 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:39:58 snowflake.connector._core Executing sync query request_id=dd51b14c-f8be-450f-9b09-b5c25894c168 20:39:58 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:39:58 snowflake.connector._core Executing sync query request_id=f5c387a4-7056-4a25-9dc6-dbfdae56c508 20:39:58 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:39:58 snowflake.connector._core Executing sync query request_id=f49df6bf-2987-48e1-b2d1-e8d677c8b2dc 20:39:58 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:39:58 snowflake.connector._core Executing sync query request_id=54d43657-f79e-4800-86c9-672d9ccf0077 20:39:58 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:39:58 snowflake.connector._core Executing sync query request_id=3fa63f75-24fa-42ed-bf9d-ccebf72da064 20:39:58 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:39:59 snowflake.connector._core Executing sync query request_id=0bd50b0f-7f75-4e2d-a290-247c12623a19 20:39:59 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:39:59 snowflake.connector._core Executing sync query request_id=33a83b22-9922-4d10-8174-2c8a709cb768 20:39:59 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:39:59 snowflake.connector._core Executing sync query request_id=d0ea7cca-5fea-49d7-a49f-0240c9a6a747 20:39:59 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:39:59 snowflake.connector._core Executing sync query request_id=3beaca59-7a86-46d5-a1e6-8b238c33e9c5 20:39:59 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:40:00 snowflake.connector._core Executing sync query request_id=719b5f9e-e6e7-4d93-bc91-e442980bd147 20:40:00 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:40:00 snowflake.connector._core Executing sync query request_id=9a2e7e94-801d-46a7-9a75-aecbd79dd1d6 20:40:00 snowflake.connector._core

outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:40:00 snowflake.connector._core Executing sync query request_id=32ecba3f-4913-4a5b-ab4e-35b463b6045b 20:40:00 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:40:00 snowflake.connector._core Executing sync query request_id=b3436a00-071a-448f-9adc-9864f4234b13 20:40:00 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:40:00 snowflake ran 00_setup.sql 20:40:00 pipeline ✓ setup (2.9s) 20:40:00 pipeline ▶ load web orders 20:40:00 snowflake.connector._core Executing sync query request_id=fd5241c6-f8b3-4348-aac4-4529a0445030 20:40:00 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:40:00 snowflake.connector._core Proxy configuration: using environment variable proxy settings 20:40:00 snowflake.connector._core S3 probe_remote_object method=HEAD bucket=sfc-th-ds1-30-customer-stage key="z8be2000-s/stages/cb06527a-f6c8-4f54-ba07-c5e78caf36a1/orders_web/2026-09-30/20260930_203955/web__20260930_203955.json.gz" 20:40:00 snowflake.connector._core outbound HTTP call method=HEAD host=sfc-th-ds1-30-customer-stage.s3.ap-southeast-7.amazonaws.com path=/z8be2000-s/stages/cb06527a-f6c8-4f54-ba07-c5e78caf36a1/orders_web/2026-09-30/20260930_203955/web__20260930_203955.json.gz 20:40:00 snowflake.connector._core HTTP response status=404 20:40:00 snowflake.connector._core outbound HTTP call method=PUT host=sfc-th-ds1-30-customer-stage.s3.ap-southeast-7.amazonaws.com path=/z8be2000-s/stages/cb06527a-f6c8-4f54-ba07-c5e78caf36a1/orders_web/2026-09-30/20260930_203955/web__20260930_203955.json.gz 20:40:00 snowflake.connector._core HTTP response status=200 20:40:00 snowflake.connector._core Executing sync query request_id=b9f25b3f-d817-481f-a999-757808000fa3 20:40:00 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:40:01 snowflake web__20260930_203955.json.gz: 7 rows copied into bronze.orders_raw 20:40:01 snowflake.connector._core Executing sync query request_id=9e5d6d20-d779-407b-9830-e2d065c2fcde 20:40:01 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:40:01 pipeline ✓ load web orders (1.0s) 20:40:01 pipeline ▶ load mobile orders 20:40:01 snowflake.connector._core Executing sync query request_id=7d7be27c-75f9-4c8b-bf54-ef8f1c8affd1 20:40:01 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:40:02 snowflake.connector._core Proxy configuration: using environment variable proxy settings 20:40:02 snowflake.connector._core S3 probe_remote_object method=HEAD bucket=sfc-th-ds1-30-customer-stage key="z8be2000-s/stages/cb06527a-f6c8-4f54-ba07-c5e78caf36a1/orders_mobile/2026-09-30/20260930_203955/mobile__20260930_203955.json.gz" 20:40:02 snowflake.connector._core outbound HTTP call method=HEAD host=sfc-th-ds1-30-customer-stage.s3.ap-southeast-7.amazonaws.com path=/z8be2000-s/stages/cb06527a-f6c8-4f54-ba07-c5e78caf36a1/orders_mobile/2026-09-30/20260930_203955/mobile__20260930_203955.json.gz 20:40:02 snowflake.connector._core HTTP response status=404 20:40:02 snowflake.connector._core outbound HTTP call method=PUT host=sfc-th-ds1-30-customer-stage.s3.ap-southeast-7.amazonaws.com path=/z8be2000-s/stages/cb06527a-f6c8-4f54-ba07-c5e78caf36a1/orders_mobile/2026-09-30/20260930_203955/mobile__20260930_203955.json.gz 20:40:02 snowflake.connector._core HTTP response status=200 20:40:02 snowflake.connector._core Executing sync query request_id=4c212b73-49ec-419b-bf81-2abd7f2138d7 20:40:02 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:40:02 snowflake mobile__20260930_203955.json.gz: 2 rows copied into bronze.orders_raw 20:40:02 snowflake.connector._core Executing sync query request_id=fdc31223-0d87-4465-8a94-218e898c66e8 20:40:02 snowflake.connector._core outbound HTTP call

method=POST path=/queries/v1/query-request attempt=1 20:40:03 pipeline ✓ load mobile orders (0.9s) 20:40:03 pipeline ▶ load crm 20:40:03 snowflake.connector._core Executing sync query request_id=53ab28e3-438e-4904-843d-aa9540364ce9 20:40:03 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:40:03 snowflake.connector._core Proxy configuration: using environment variable proxy settings 20:40:03 snowflake.connector._core S3 probe_remote_object method=HEAD bucket=sfc-th-ds1-30-customer-stage key="z8be2000-s/stages/cb06527a-f6c8-4f54-ba07-c5e78caf36a1/crm/2026-09-30/20260930_203955/crm__20260930_203955.json.gz" 20:40:03 snowflake.connector._core outbound HTTP call method=HEAD host=sfc-th-ds1-30-customer-stage.s3.ap-southeast-7.amazonaws.com path=/z8be2000-s/stages/cb06527a-f6c8-4f54-ba07-c5e78caf36a1/crm/2026-09-30/20260930_203955/crm__20260930_203955.json.gz 20:40:03 snowflake.connector._core HTTP response status=404 20:40:03 snowflake.connector._core outbound HTTP call method=PUT host=sfc-th-ds1-30-customer-stage.s3.ap-southeast-7.amazonaws.com path=/z8be2000-s/stages/cb06527a-f6c8-4f54-ba07-c5e78caf36a1/crm/2026-09-30/20260930_203955/crm__20260930_203955.json.gz 20:40:03 snowflake.connector._core HTTP response status=200 20:40:03 snowflake.connector._core Executing sync query request_id=b41f487a-aed9-43b2-9292-dbfed210c1b7 20:40:03 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:40:03 snowflake crm__20260930_203955.json.gz: 5 rows copied into bronze.customers_raw 20:40:03 snowflake.connector._core Executing sync query request_id=af1a4803-1fd4-4d21-b98c-33aece058980 20:40:03 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:40:04 pipeline ✓ load crm (0.7s) 20:40:04 pipeline ▶ load fx 20:40:04 snowflake.connector._core Executing sync query request_id=bc94ea1c-17f3-4b78-8bb7-bef7938ffaec 20:40:04 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:40:04 snowflake.connector._core Proxy configuration: using environment variable proxy settings 20:40:04 snowflake.connector._core S3 probe_remote_object method=HEAD bucket=sfc-th-ds1-30-customer-stage key="z8be2000-s/stages/cb06527a-f6c8-4f54-ba07-c5e78caf36a1/fx/2026-09-30/20260930_203955/fx__20260930_203955.json.gz" 20:40:04 snowflake.connector._core outbound HTTP call method=HEAD host=sfc-th-ds1-30-customer-stage.s3.ap-southeast-7.amazonaws.com path=/z8be2000-s/stages/cb06527a-f6c8-4f54-ba07-c5e78caf36a1/fx/2026-09-30/20260930_203955/fx__20260930_203955.json.gz 20:40:04 snowflake.connector._core HTTP response status=404 20:40:04 snowflake.connector._core outbound HTTP call method=PUT host=sfc-th-ds1-30-customer-stage.s3.ap-southeast-7.amazonaws.com path=/z8be2000-s/stages/cb06527a-f6c8-4f54-ba07-c5e78caf36a1/fx/2026-09-30/20260930_203955/fx__20260930_203955.json.gz 20:40:04 snowflake.connector._core HTTP response status=200 20:40:04 snowflake.connector._core Executing sync query request_id=96f43093-f868-4c76-a4d5-6d2a7c86e4c8 20:40:04 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:40:04 snowflake fx__20260930_203955.json.gz: 1 rows copied into bronze.fx_raw 20:40:04 snowflake.connector._core Executing sync query request_id=743bcf01-bad2-40cd-b617-5f80511dee8a 20:40:04 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:40:05 pipeline ✓ load fx (0.7s) 20:40:05 pipeline ▶ 10 cleanse orders 20:40:05 snowflake.connector._core Executing sync query request_id=d9638ddb-baf6-4888-bb97-5b0f101cca64 20:40:05 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:40:05 snowflake.connector._core Executing sync query request_id=45539d05-709b-48f3-8684-fe46b036206a 20:40:05 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:40:05 snowflake.connector._core Executing sync query request_id=13668e7a-cb3e-4651-8827-

c2ce5457bc6c 20:40:05 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:40:05 snowflake.connector._core Executing sync query request_id=6f29dd6d-9256-4eb5-9cf0-70b8c856fe81 20:40:05 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:40:06 snowflake.connector._core Executing sync query request_id=187d904d-0284-46f0-b87c-5c5cbaf917e1 20:40:06 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:40:06 snowflake ran 10_silver_orders_clean.sql 20:40:06 snowflake.connector._core Executing sync query request_id=1ab60309-46d1-4e80-8b88-99d9dbbeb879 20:40:06 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:40:06 pipeline ✓ 10 cleanse orders (1.6s) 20:40:06 pipeline ▶ 20 deduplicate 20:40:06 snowflake.connector._core Executing sync query request_id=33ccd683-6c5a-411c-979b-2469799e71af 20:40:06 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:40:06 snowflake.connector._core Executing sync query request_id=2312cba8-d22b-4eb1-bd88-556be24ee043 20:40:06 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:40:07 snowflake.connector._core Executing sync query request_id=5c17ebac-26ba-4e76-a640-9453654d6c68 20:40:07 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:40:07 snowflake ran 20_silver_orders_dedup.sql 20:40:07 snowflake.connector._core Executing sync query request_id=01919228-f4fc-43e0-919d-370bed21397e 20:40:07 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:40:07 pipeline ✓ 20 deduplicate (0.6s) 20:40:07 snowflake.connector._core Executing sync query request_id=48cbdc16-e387-4c6a-97f5-dc5b96112dd0 20:40:07 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:40:08 pipeline new pincodes to look up: [] 20:40:08 extract pincode: 0 records, nothing written 20:40:08 pipeline ▶ load pincodes 20:40:08 snowflake.connector._core Executing sync query request_id=88bb16cc-643f-4967-811a-6d4d42a467cf 20:40:08 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:40:08 pipeline ✓ load pincodes (0.0s) 20:40:08 pipeline ▶ 30 customers scd2 20:40:08 snowflake.connector._core Executing sync query request_id=240d489b-252c-423f-8bef-e5107c438000 20:40:08 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:40:08 snowflake.connector._core Executing sync query request_id=1f80eea3-a4af-437d-a320-c4af2a4dc7cb 20:40:08 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:40:08 snowflake.connector._core Executing sync query request_id=dbc6acbd-00de-4414-9256-8a8393123531 20:40:08 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:40:09 snowflake ran 30_dim_customers_scd2.sql 20:40:09 snowflake.connector._core Executing sync query request_id=c9b51f62-b4a6-4ffa-864f-0b5b301be11a 20:40:09 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:40:09 pipeline ✓ 30 customers scd2 (0.7s) 20:40:09 pipeline ▶ 31 fx daily 20:40:09 snowflake.connector._core Executing sync query request_id=552ce2c5-1225-4e68-b3c8-4c5a71e9a359 20:40:09 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:40:09 snowflake.connector._core Executing sync query request_id=9b8de9ed-9839-44ec-81f0-3630dcebbd0e 20:40:09 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:40:09 snowflake.connector._core Executing sync query request_id=e3228dbf-d79f-416f-89b7-377cd3d97464 20:40:09 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:40:10 snowflake ran

31_ref_fx_daily.sql 20:40:10 snowflake.connector._core Executing sync query request_id=0dd79425-dc35-4ab2-a690-d8651fd53b14 20:40:10 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:40:10 pipeline ✓ 31 fx daily (0.8s) 20:40:10 pipeline ▶ 32 pincode geo 20:40:10 snowflake.connector._core Executing sync query request_id=03032a10-230a-4655-b631-2bd9d646d3a6 20:40:10 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:40:10 snowflake.connector._core Executing sync query request_id=ab0526fe-1907-401b-b549-8812c60611d3 20:40:10 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:40:10 snowflake.connector._core Executing sync query request_id=4919041f-a07b-4e59-bdba-dee37e7b0401 20:40:10 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:40:10 snowflake ran 32_ref_pincode_geo.sql 20:40:11 snowflake.connector._core Executing sync query request_id=26f6a28a-fd60-44e5-beb8-cd58c5188d39 20:40:11 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:40:11 pipeline ✓ 32 pincode geo (0.7s) 20:40:11 pipeline ▶ 40 enrich + gold 20:40:11 snowflake.connector._core Executing sync query request_id=807528ab-0468-46aa-843e-8d963646796a 20:40:11 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:40:11 snowflake.connector._core Executing sync query request_id=00799d72-919c-4852-a8a1-86c837beaae5 20:40:11 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:40:11 snowflake.connector._core Executing sync query request_id=e018b1e3-d8ca-47e0-8128-1804378c8aab 20:40:11 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:40:12 snowflake.connector._core Executing sync query request_id=b78f8642-88ee-48b5-8062-4f62ac60f47e 20:40:12 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:40:12 snowflake ran 40_gold_fct_orders.sql 20:40:12 snowflake.connector._core Executing sync query request_id=f499e4f8-18ec-46a3-ab5b-c872cb065808 20:40:12 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:40:12 pipeline ✓ 40 enrich + gold (1.3s) 20:40:12 pipeline ▶ quality checks 20:40:12 snowflake.connector._core Executing sync query request_id=088fb0e0-0d74-4d5b-b98f-1dd93a348036 20:40:12 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:40:12 pipeline no fan-out in enrichment (rows out - rows in) 0 OK 20:40:12 snowflake.connector._core Executing sync query request_id=c24975c6-c0db-4b77-b6b0-573cdf3e9033 20:40:12 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:40:12 pipeline one gold row per order_id (duplicates) 0 OK 20:40:12 snowflake.connector._core Executing sync query request_id=695db336-c60d-40b2-af3d-67ac9832c867 20:40:12 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:40:13 pipeline orders with no customer match 1 WARN 20:40:13 snowflake.connector._core Executing sync query request_id=8066ca0d-ab9e-4596-835a-e8248b377f25 20:40:13 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:40:13 pipeline orders with an amount but no FX rate 1 WARN 20:40:13 snowflake.connector._core Executing sync query request_id=720b31e9-3557-4c3f-9019-ed912fd5b9df 20:40:13 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:40:13 pipeline orders with no city 0 OK 20:40:13 snowflake.connector._core Executing sync query request_id=7a6e5d52-dc89-44d7-96fe-998772a0589d 20:40:13 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:40:13 pipeline orders with future order_date 1 FAIL 20:40:13 snowflake.connector._core Executing sync query request_id=dfe6d725-441b-4cda-9b97-

63b56e280bf8 20:40:13 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:40:13 pipeline quarantine rate (max 5%) 0.242424 WARN 20:40:13 snowflake.connector._core Executing sync query request_id=722a8836-8a93-48d8-879b-57b52c424bf2 20:40:13 snowflake.connector._core outbound HTTP call method=POST path=/queries/v1/query-request attempt=1 20:40:13 pipeline ✕ quality checks failed: quality checks failed: orders with future order_date = 1 20:40:13 snowflake.connector._core Closing connection 20:40:13 snowflake.connector._core Sending logout: auto-detection found no running async queries 20:40:13 snowflake.connector._core Initiating session logout 20:40:13 snowflake.connector._core outbound HTTP call method=POST path=/session attempt=1 20:40:13 snowflake.connector._core Session logout completed successfully 20:40:13 snowflake.connector._core Logout completed successfully 20:40:13 snowflake.connector._core Connection closed successfully 20:40:13 pipeline pipeline failed: quality checks failed: orders with future order_date = 1

C1

Block Name	Block Type	Functionality	Attached Test
load_iris	data_loader	Loads the Iris dataset (150 records).	test_output()
prepare_data	transformer	Splits data into 80% training and 20% testing sets	test_output()
train_model	transformer	Trains a Decision Tree classifier and calculates accuracy.	test_output()
save_results	data_exporter	Saves actual and predicted values into a CSV file	test_output()

The following command was used to execute the pipeline along with its tests

.\.venv\Scripts\mage.exe run iris_project iris_classifier –test

The Mage pipeline executed successfully, and all four block tests passed. The Decision Tree classifier achieved an accuracy of 96.67%. The actual and predicted values were saved in iris_predictions.csv

Screenshots in the appendix

C2

max_depth	Test accuracy	Test passed?	CSV created?
1	66.67%	No	No
2	93.33%	Yes	Yes
3	96.67%	Yes	Yes
4	93.33%	Yes	Yes
5	93.33%	Yes	Yes

The @test function in the train_model block acts as a gate before the next block executes. It checks whether the model's test accuracy is at least 80%. When max_depth = 1, the accuracy is only 66.67%, so the test fails. Mage stops the pipeline, and the save_results block does not execute. Therefore, iris_predictions.csv is not generated.

Why is this better than checking after saving?

It prevents incorrect or low-quality results from being exported. The problem is detected immediately, before the output file is created. This makes the pipeline more reliable and avoids saving results that do not meet the required accuracy.

C3

Metadata.yaml screenshot in appendix

Magepipeline UI cloudflare in appendix

Confusion matrix and metrics table output

[[10 0 0]

[0 9 1]

[0 1 9]]

Metric	Score
Accuracy	0.933333
Precision	0.933333

Recall 0.933333

F1-score 0.933333

The pipeline was executed using:

.\.venv\Scripts\mage.exe run iris_project iris_classifier --test

C4. Your Own Block – Data Validation

For this task, I created a new transformer block named validate_data.py in the Mage project.

The block is placed after the load_iris block and before the prepare_data block.

Purpose:
The purpose of this block is to validate the Iris dataset before further processing.

Validations performed:

1. Checks whether the dataset contains any missing values.
2. Checks whether the target column contains exactly 3 unique classes.

Testing:
I added a separate @test function to the block to verify that:

- The output is not empty.
- There are no missing values.
- Exactly 3 unique classes are present.

If the validation conditions are satisfied, the block returns the validated dataset for the next stage of the pipeline.

Result:

The validation block was added successfully to the Mage pipeline. The pipeline was tested locally, and the GitHub Actions CI workflow also passed successfully after committing and pushing the changes.

Conclusion:

This block ensures that the Iris dataset meets the required conditions before proceeding to data preparation and model training

D1

No.	Routing key	Receiving queues
1	bangalore.koramangala.food	Ravi, Meena, Control
2	bangalore.indiranagar.ride	Priya, Arjun, Control
3	bangalore.whitefield.ride	Arjun, Control
4	chennai.tnagar.food	Karthik
5	chennai.whitefield.ride	Dropped
6	bangalore.koramangala	Control

The routing key bangalore.koramangala contains only two words.

- Ravi: Does not match because * requires exactly one additional word.
- Priya: Does not match because the second word is different.
- Arjun: Does not match because the pattern requires three words ending in ride.
- Meena: Does not match because the pattern requires three words ending in food.
- Karthik: Does not match because the first word is different.
- Control: Matches because # can match zero or more additional words.

Therefore, only the Control queue receives the message.

D2 – Build It: RabbitMQ Topic Exchange

Objective

To implement a message dispatcher and multiple drivers using Python and the Pika library with a RabbitMQ topic exchange.

Implementation

1. Dispatcher (dispatcher.py)

- Connects to the RabbitMQ server.
- Declares a topic exchange named rapido-dispatch.

- Randomly selects a city (Bangalore or Chennai), zone, and request type (ride or food).
- Generates a routing key in the format city.zone.type.
- Creates a JSON message containing a unique ID, city, zone, type, and timestamp.
- Publishes a new message every 2 seconds.

2. Driver (driver.py)

- Accepts the driver's name and routing pattern as command-line arguments.
- Creates a private, exclusive queue for each driver.
- Binds the queue to the rapido-dispatch exchange using the given routing pattern.
- Receives and displays matching requests.
- Sends an acknowledgement (ACK) after processing each message.

Testing and Results

The dispatcher and three drivers (Ravi, Meena, and Control) were executed using RabbitMQ in Google Colab for 2 minutes.

The following routing patterns were used:

Driver

Binding pattern

Ravi

bangalore.koramangala.*

Meena

bangalore.*.food

Control

bangalore.#

Each driver received 8 messages during the test.

A message with routing key bangalore.koramangala.food was received by all three drivers. This matches the routing results predicted in Part D1:

- Ravi received it because the zone was Koramangala.
- Meena received it because the request type was food.
- Control received it because # matches zero or more words after Bangalore.

Conclusion

The RabbitMQ topic exchange successfully routed messages to drivers based on their binding patterns. The two-minute test verified the routing behaviour, and all three drivers successfully received and acknowledged their matching messages.

D3

ACK and NACK Handling

Objective

To implement message acknowledgement (ACK) and rejection (NACK) in RabbitMQ and understand how invalid messages are handled.

1. Driver Implementation

The driver checks whether the received message contains the zone field.

- If zone is present, the driver processes the message and sends basic_ack().
- If zone is missing, the driver rejects the message using basic_nack() with requeue=False.

This ensures that invalid messages are not repeatedly delivered.

2. Dispatcher Implementation

The dispatcher publishes messages every 2 seconds. It deliberately removes the zone field from every 10th message to simulate invalid data.

3. Testing and Results

The dispatcher was executed for 2 minutes, publishing 60 messages.

Six messages were intentionally published without the zone field. The three drivers received messages according to their routing patterns.

Driver	Binding pattern	Invalid messages rejected
Ravi	bangalore.koramangala.*	2
Meena	bangalore.*.food	3
Control	bangalore.#	4

The logs showed both PROCESSED and REJECTED messages. This confirms that valid messages were acknowledged and invalid messages were rejected.

What happens if a driver crashes before sending ACK?

RabbitMQ considers the message unacknowledged. When the consumer's connection closes, RabbitMQ makes the message available for redelivery to another consumer, provided the queue still exists.

What happens if we use requeue=True for an always-bad message?

RabbitMQ puts the rejected message back into the queue. The driver may receive and reject it repeatedly, creating an infinite requeue loop. Using requeue=False prevents this repeated delivery.

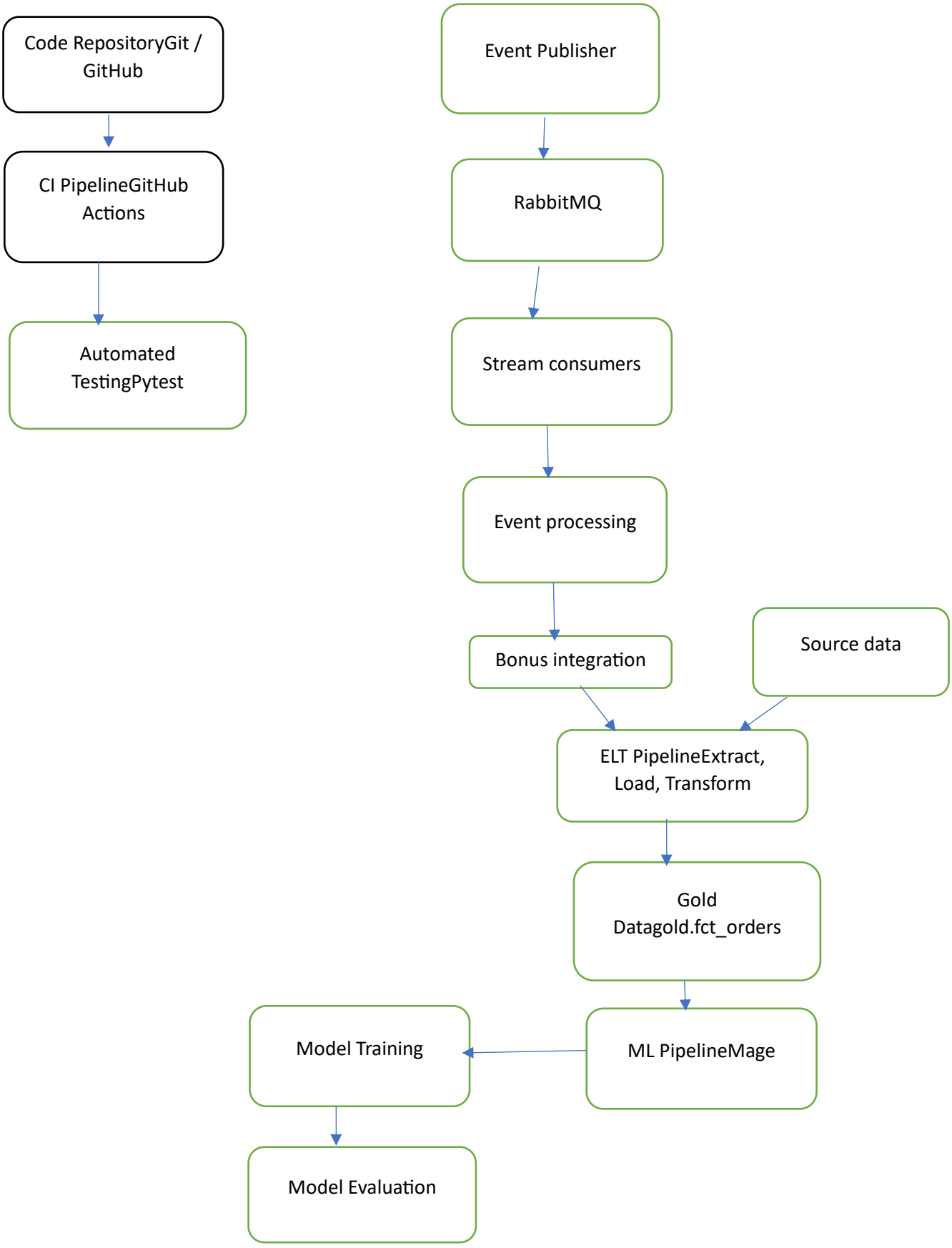
Conclusion

Successfully implemented ACK and NACK handling using RabbitMQ. The test demonstrated that valid messages are processed while invalid messages are rejected without requeueing.

ALL SCREENSHOTS ARE THERE IN APPENDIX FROM A TO D

PART E

Q2



GitHub for code management and CI testing to verify code quality. The ELT pipeline processes source data and produces Gold data. The ML pipeline uses data for model training and evaluation. Meanwhile, RabbitMQ handles event streaming through publishers and consumers. The bonus integration connects streamed orders to the ELT pipeline.

Q3 Reflection

Working on this project was a valuable learning experience because it gave me an opportunity to understand how different data engineering concepts work together in a practical environment. Initially, I faced difficulties setting up the project, understanding the tools and executing the different components. Since the project involved Git, GitHub Actions, Mage, RabbitMQ and Snowflake, understanding how each component contributed to the overall workflow took time.

One of the challenges I faced was implementing the CI workflow. During testing, I temporarily changed the test_size parameter from 0.2 to 0.3, which caused the test to fail. By examining the test output, I understood that the expected training and testing records did not match the actual split. I corrected the parameter and verified that the workflow passed successfully in GitHub Actions.

In the ML pipeline, I experimented with different decision tree depths and compared their performance using accuracy, precision, recall and F1-score. This helped me understand the importance of model evaluation rather than depending only on the training process. While implementing RabbitMQ, I learned how messages are routed to different consumers and how acknowledgements are used to handle successful and unsuccessful message processing.

The bonus task of connecting RabbitMQ with the ELT pipeline was particularly challenging because it required integrating message streaming with data storage and transformation. I collected order messages, loaded them into Snowflake, performed the necessary transformations and merged them into the Gold table. I also verified idempotency by running the operation twice and confirming that the Gold totals remained unchanged.

Overall, this project improved my practical understanding of version control, automated testing, data processing, machine learning and stream processing. It also helped me become more comfortable working with different tools and troubleshooting problems during implementation.

Bonus: Connecting Part D (RabbitMQ) to Part B (ELT Pipeline)

A RabbitMQ consumer was implemented to receive FreshCart-style order messages. The consumer collected messages for one minute and received three sample orders: ORD-2001, ORD-2002 and ORD-2003.

The received messages were saved as a compressed JSON Lines file in the landing zone using a write_landing() implementation. The file was then uploaded to Snowflake and loaded into bronze.orders_raw using the equivalent of the Part B load_file() operation.

Next, the orders were cleaned, deduplicated and enriched in the Silver layer using SQL transformations. Customer details, currency conversion rates, loyalty points and location information were added.

Finally, the three streamed orders were merged into gold.fct_orders.

Idempotency verification:

Metric	Before replay	After replay
Total orders	7	7
Total amount (INR)	9935.00	9935.00
Total loyalty points	212	212

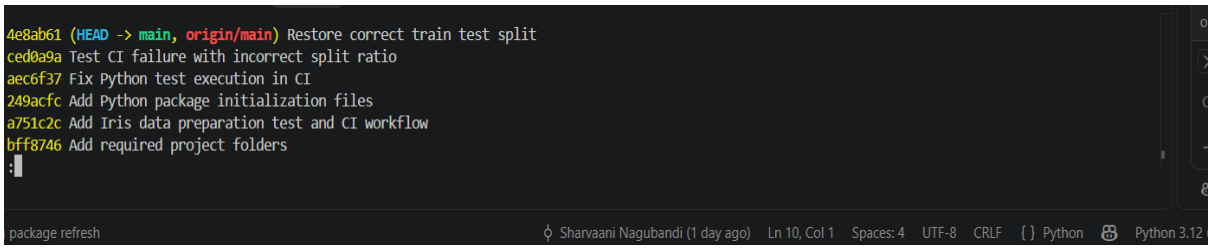
The second execution did not create duplicate orders or change the totals. Therefore, the idempotency check passed.

APPENDIX

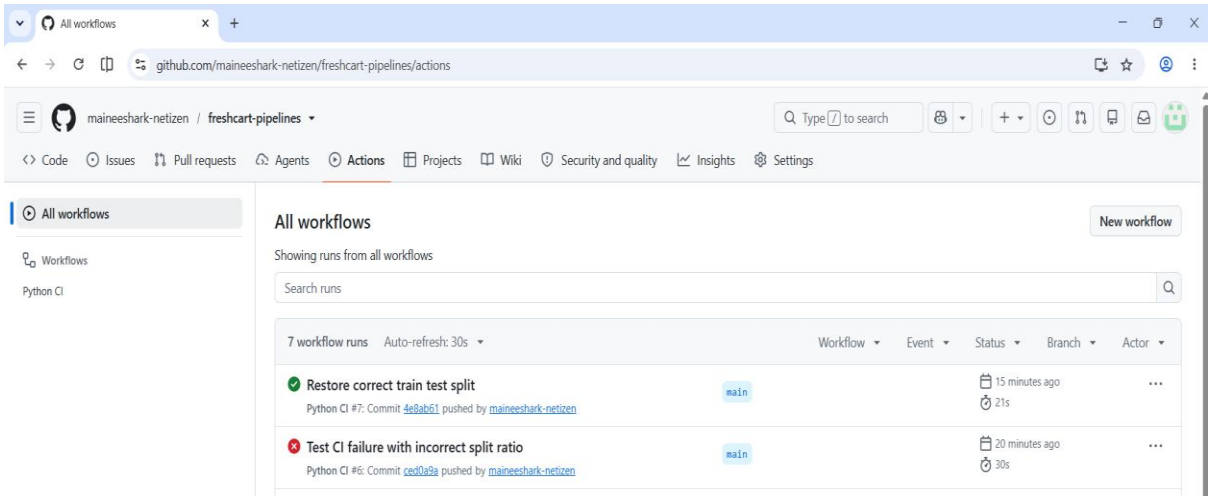
[ALL SCREENSHOTS INCLUDED FROM PART A TO D]

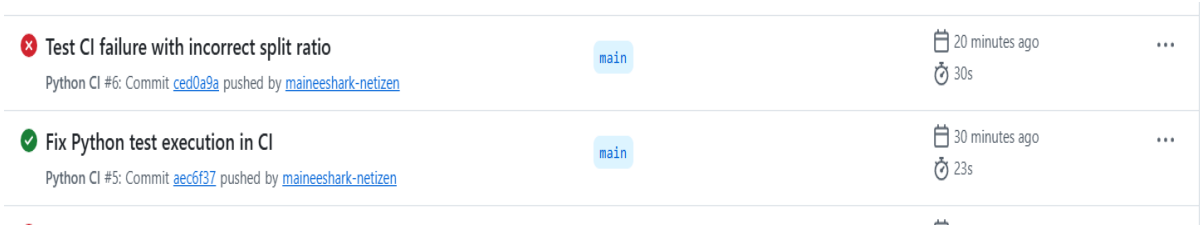
BONUS SCREENSHOTS ALSO INCLUDED AFTER PART D

A1



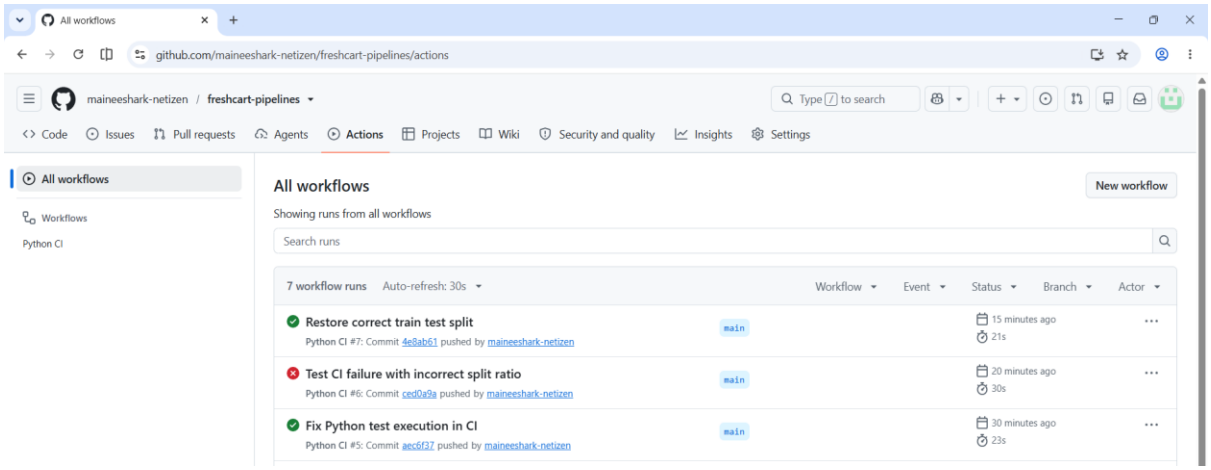
A3 a screenshot of a green (passing) run in the GitHub Actions tab



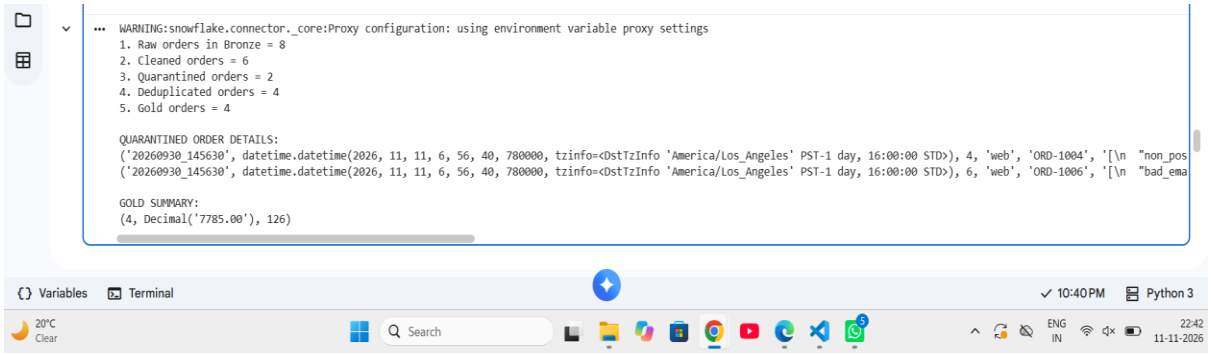


a screenshot of a red (failing) run, made by temporarily changing test_size in prepare_data.py to 0.3, followed by the commit that fixes it

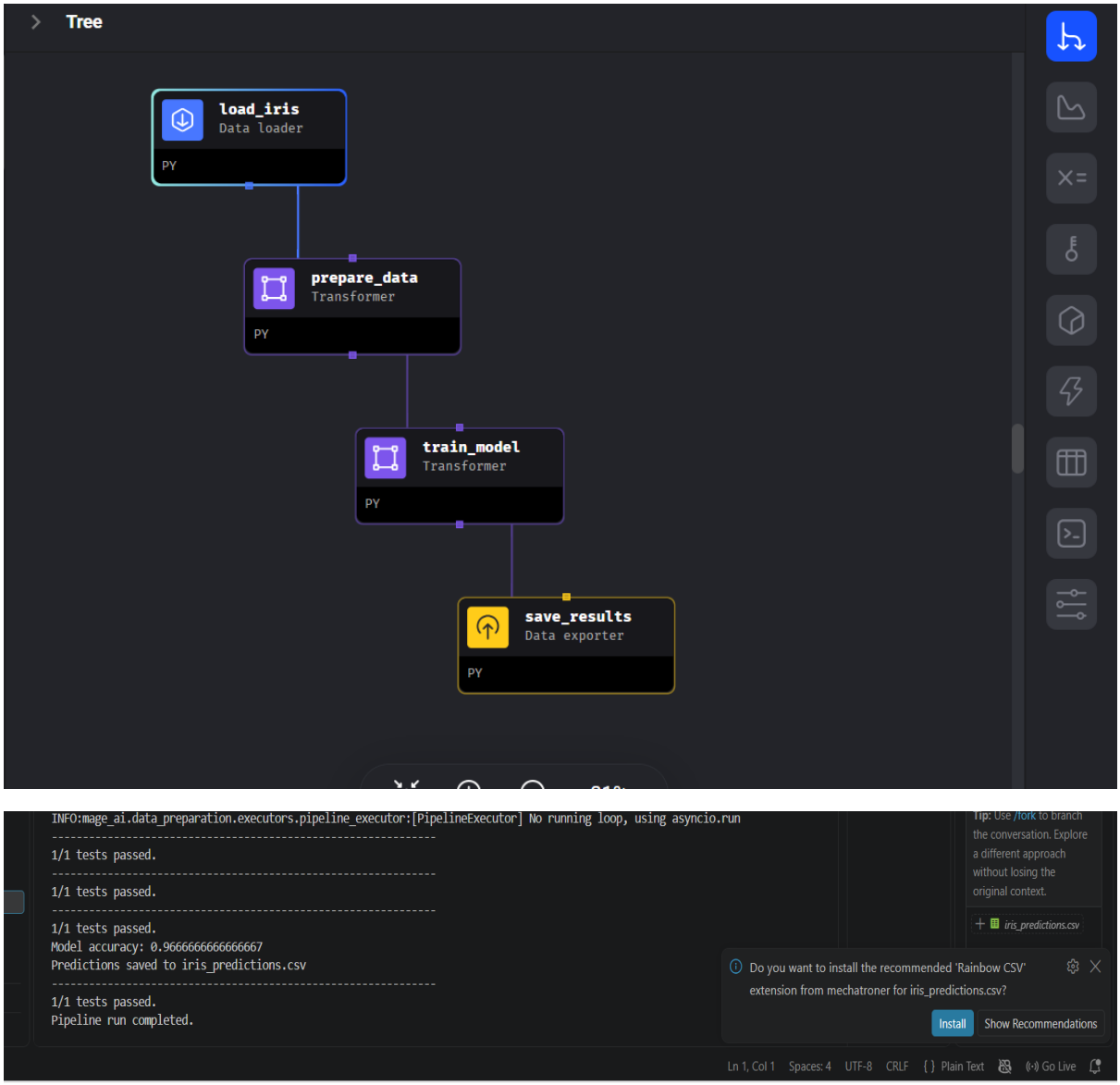
Im keeping whole screenshot for A3 for understanding for passing the test,test for failure and again restoring to correct train split



B1

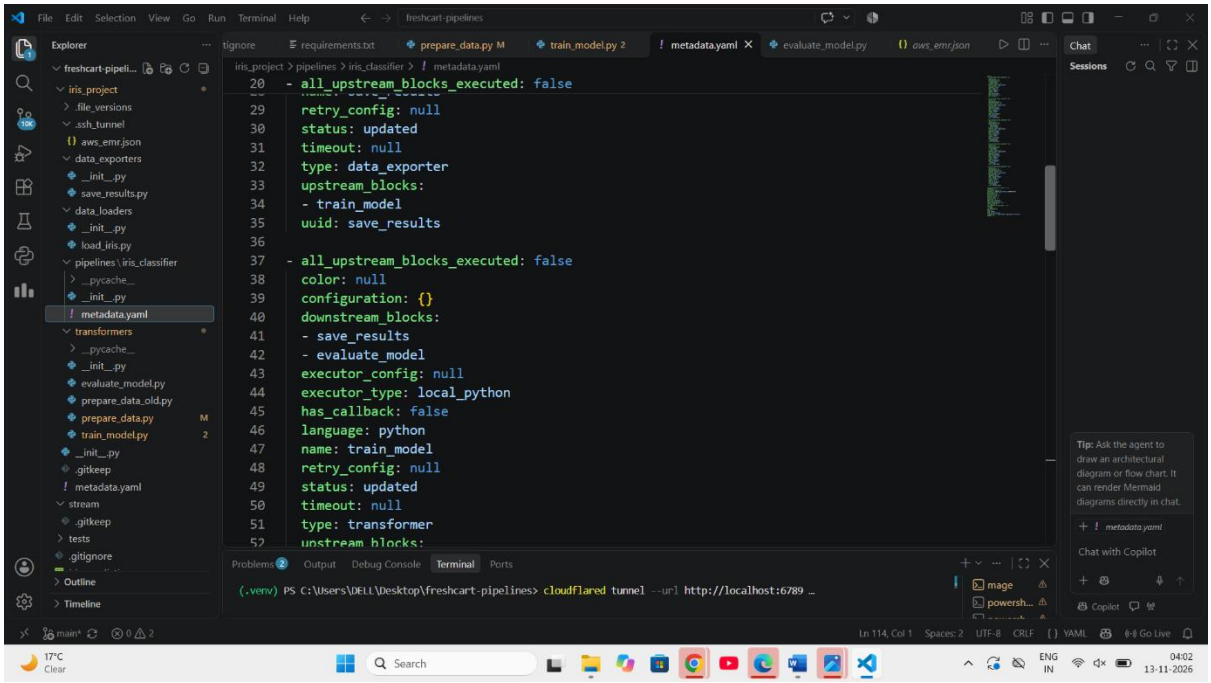


C1

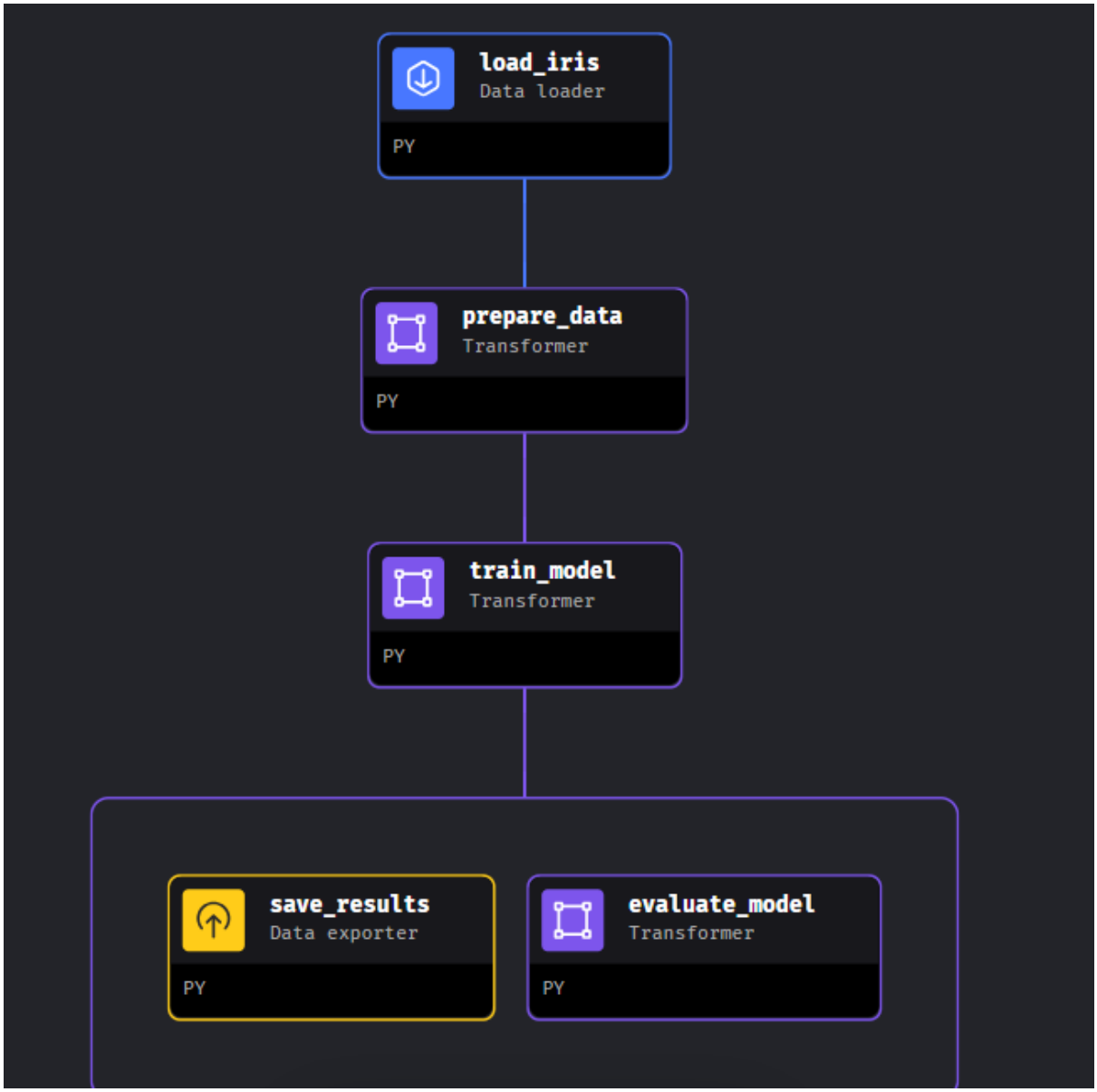


The Mage pipeline executed successfully, and all four block tests passed. The Decision Tree classifier achieved an accuracy of 96.67%. The actual and predicted values were saved in iris_predictions.csv

C3 metadata.yaml



Magepipeline UI Cloudflare

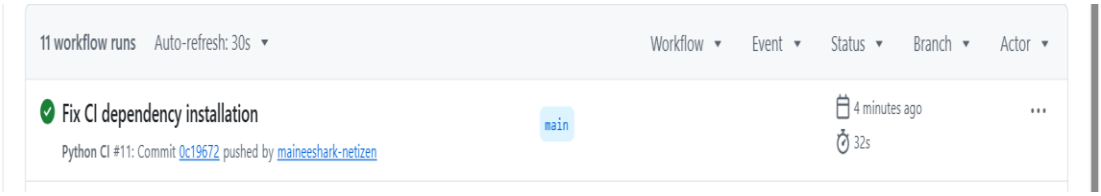


Confusion matrix and metrics table output

```
----------------------------------------------------------------------
1/1 tests passed.
Max depth: 5
Test accuracy: 0.9333333333333333
----------------------------------------------------------------------
1/1 tests passed.
Confusion Matrix:
[[10  0  0]
 [ 0  9  1]
 [ 0  1  9]]

Evaluation Metrics:
   Metric    Score
 Accuracy 0.933333
Precision 0.933333
   Recall 0.933333
 F1-score 0.933333
Model accuracy: 0.9333333333333333
Predictions saved to iris_predictions.csv
----------------------------------------------------------------------
1/1 tests passed.
----------------------------------------------------------------------
1/1 tests passed.
Pipeline run completed.
```

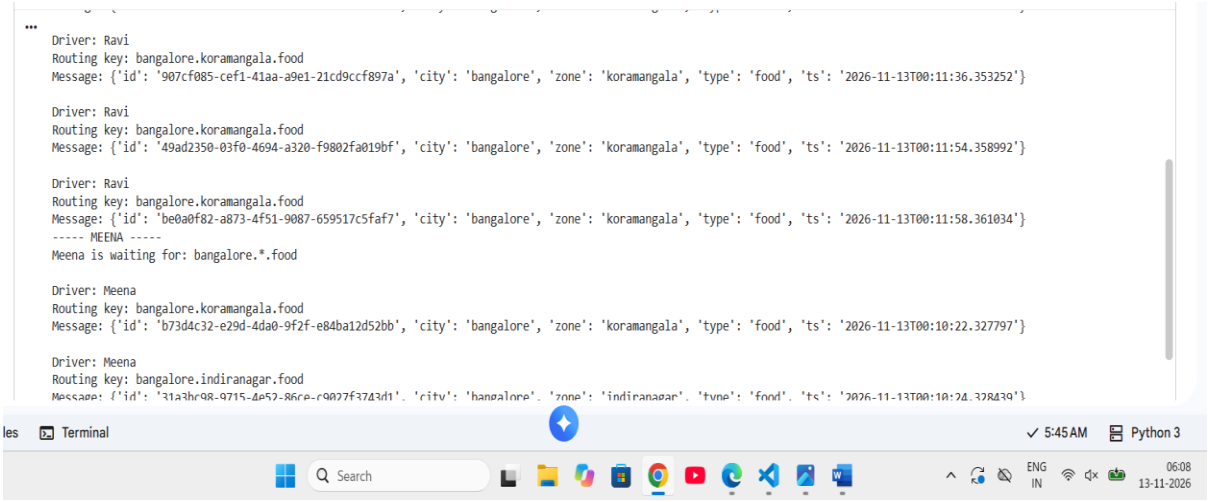
C4

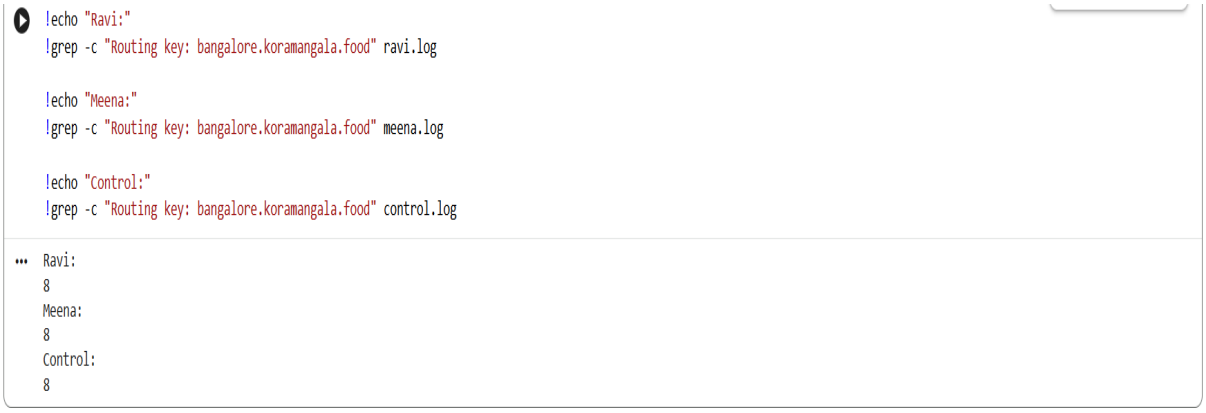


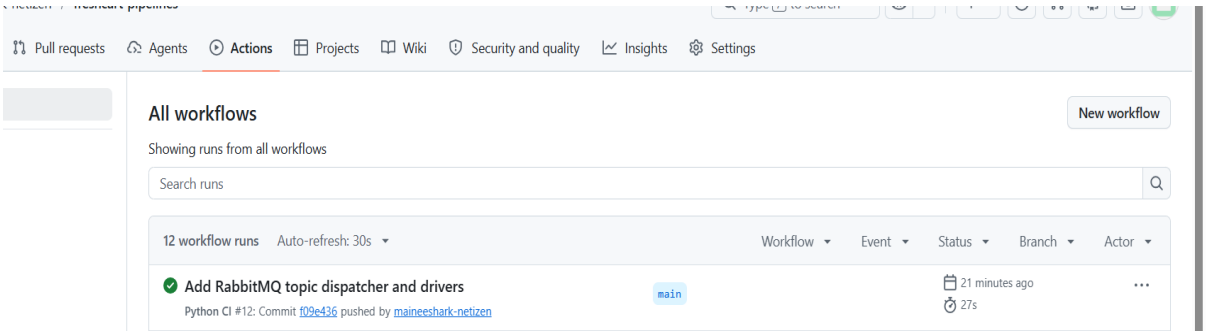
D2

```
!cat dispatcher.log
```

```
shed: bangalore.koramangala.food {'id': '2e3014e5-1a26-4e71-b8b0-dc72aeb4d03c', 'city': 'bangalore', 'zone': 'koramangala', 'type': 'food', 'ts': '2026-11-13T00:10:26.329005
shed: bangalore.koramangala.food {'id': 'aeeab85b-2b6f-4be1-a30d-b4f8aec22ab7', 'city': 'bangalore', 'zone': 'koramangala', 'type': 'food', 'ts': '2026-11-13T00:10:28.329594
shed: bangalore.indiranagar.food {'id': 'bf426ee3-cc81-4819-b797-58b84992346a', 'city': 'bangalore', 'zone': 'indiranagar', 'type': 'food', 'ts': '2026-11-13T00:10:30.330137
shed: bangalore.indiranagar.ride {'id': 'a2faf237-2db4-4388-93a7-419b617c807d', 'city': 'bangalore', 'zone': 'indiranagar', 'type': 'ride', 'ts': '2026-11-13T00:10:32.330724
shed: chennai.whitefield.food {'id': '226b9be8-b233-4a4b-9820-209995481396', 'city': 'chennai', 'zone': 'whitefield', 'type': 'food', 'ts': '2026-11-13T00:10:34.331336'}
shed: bangalore.whitefield.food {'id': 'afe27517-16a7-41a9-81d5-b57850fc1d5a', 'city': 'bangalore', 'zone': 'whitefield', 'type': 'food', 'ts': '2026-11-13T00:10:36.331923'}
shed: chennai.tnagar.food {'id': 'b415f2fa-57f5-47b1-9851-6cccea8abd70', 'city': 'chennai', 'zone': 'tnagar', 'type': 'food', 'ts': '2026-11-13T00:10:38.332550'}
shed: bangalore.koramangala.ride {'id': 'b7d97798-8641-4106-beac-2da6b0a55516', 'city': 'bangalore', 'zone': 'koramangala', 'type': 'ride', 'ts': '2026-11-13T00:10:40.333127
shed: chennai.whitefield.ride {'id': 'c7885cb1-4117-474b-b7a5-3b107e5baebb', 'city': 'chennai', 'zone': 'whitefield', 'type': 'ride', 'ts': '2026-11-13T00:10:42.333701'}
shed: bangalore.indiranagar.food {'id': 'c7a160ad-778c-4ce7-94c7-185007bc4d1a', 'city': 'bangalore', 'zone': 'indiranagar', 'type': 'food', 'ts': '2026-11-13T00:10:44.334332
shed: chennai.whitefield.ride {'id': '84f66fe2-19eb-4528-b191-06ef0cb29de9', 'city': 'chennai', 'zone': 'whitefield', 'type': 'ride', 'ts': '2026-11-13T00:10:46.335034'}
shed: bangalore.whitefield.ride {'id': 'd718255a-e44e-484c-957b-73bf56f8319d', 'city': 'bangalore', 'zone': 'whitefield', 'type': 'ride', 'ts': '2026-11-13T00:10:48.335698'}
shed: bangalore.koramangala.ride {'id': '2ea45a08-efa8-42f1-acc7-35f5fd08e336', 'city': 'bangalore', 'zone': 'koramangala', 'type': 'ride', 'ts': '2026-11-13T00:10:50.336264
shed: chennai.whitefield.food {'id': 'e76397e6-d386-456b-a3e8-eb80f8b1ed83', 'city': 'chennai', 'zone': 'whitefield', 'type': 'food', 'ts': '2026-11-13T00:10:52.337081'}
shed: bangalore.whitefield.ride {'id': 'c0c1de5a-923f-4f4e-9cf1-8b6082a674a2', 'city': 'bangalore', 'zone': 'whitefield', 'type': 'ride', 'ts': '2026-11-13T00:10:54.337695'}
shed: chennai.whitefield.ride {'id': '1eb69b97-51e2-4c98-8b75-7a94fa650683', 'city': 'chennai', 'zone': 'whitefield', 'type': 'ride', 'ts': '2026-11-13T00:10:56.338276'}
shed: bangalore.indiranagar.food {'id': 'f47bcac6-45c4-449d-919d-cb92ce3507d6', 'city': 'bangalore', 'zone': 'indiranagar', 'type': 'food', 'ts': '2026-11-13T00:10:58.338954
shed: bangalore.whitefield.ride {'id': 'a319c77d-75a3-436c-8128-28bbb05fa268', 'city': 'bangalore', 'zone': 'whitefield', 'type': 'ride', 'ts': '2026-11-13T00:11:00.341388'}
shed: chennai.tnagar.ride {'id': 'd826c647-fdd0-4618-a003-23312f15e669', 'city': 'chennai', 'zone': 'tnagar', 'type': 'ride', 'ts': '2026-11-13T00:11:02.342044'}
shed: chennai.tnagar.food {'id': '0a9dddb2-b7b7-4ec0-a7af-b99ce017c248', 'city': 'chennai', 'zone': 'tnagar', 'type': 'food', 'ts': '2026-11-13T00:11:04.342684'}
shed: bangalore.indiranagar.food {'id': 'd93e408b-abf1-452a-9b64-b9050bd292bd', 'city': 'bangalore', 'zone': 'indiranagar', 'type': 'food', 'ts': '2026-11-13T00:11:06.343343
shed: bangalore.whitefield.food {'id': '85da8271-bcb8-49d2-ba7b-a31577d9a4bf', 'city': 'bangalore', 'zone': 'whitefield', 'type': 'food', 'ts': '2026-11-13T00:11:08.343963'}
shed: chennai.whitefield.ride {'id': 'c607f8e9-67d3-4de3-ae10-0769c46930c7', 'city': 'chennai', 'zone': 'whitefield', 'type': 'ride', 'ts': '2026-11-13T00:11:10.344638'}
shed: bangalore.whitefield.food {'id': 'acb74186-8ca8-4774-953b-6d8e3af4ba8e', 'city': 'bangalore', 'zone': 'whitefield', 'type': 'food', 'ts': '2026-11-13T00:11:12.345316'}
```







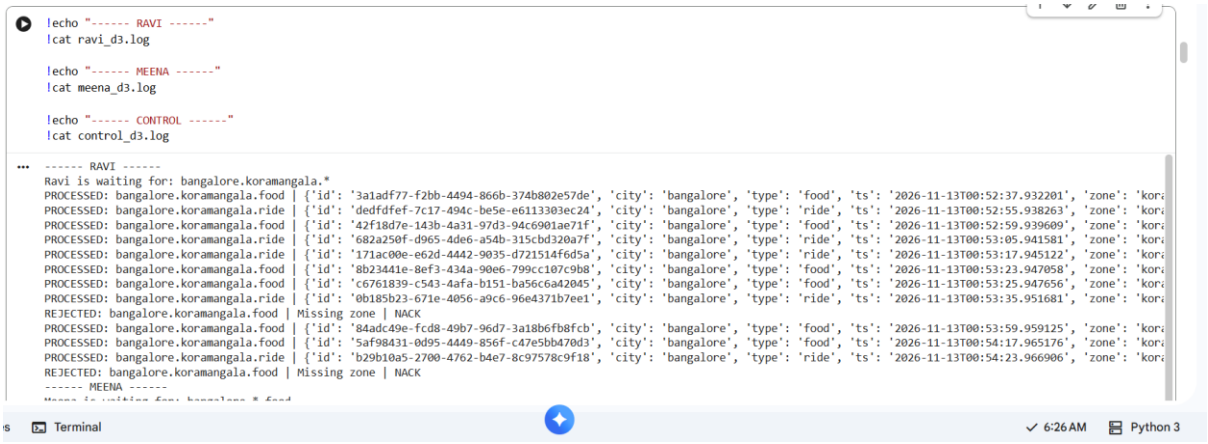
D3

RabbitMQ considers the message unacknowledged. When the connection closes, RabbitMQ puts the message back into the queue so that it can be delivered again to another available consumer. This is called redelivery.

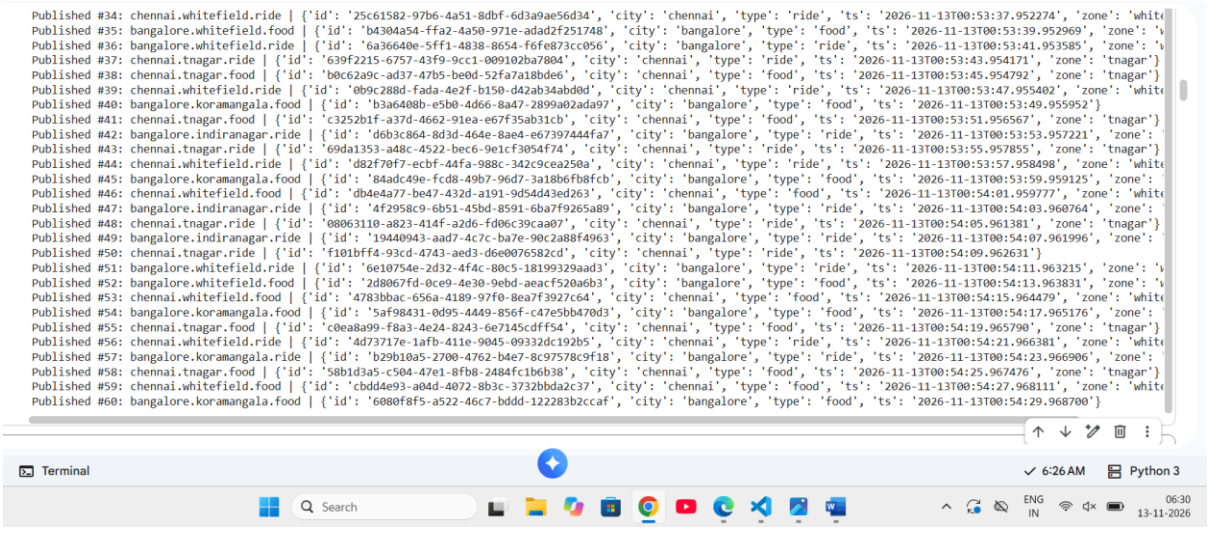
RabbitMQ puts the rejected message back into the queue. The driver receives it again, rejects it again, and the cycle repeats. This can create an infinite message requeue loop.

That's why we use requeue=False for permanently invalid messages





BONUS



[40]
✓ 0s

```
cur.execute("SELECT COUNT(*) FROM gold.fct_orders")
print("Existing Gold orders:", cur.fetchone()[0])
```

```
Existing Gold orders: 4
```

[41]
✓ 0s

```
import gzip, json
from pathlib import Path

cur.execute("""
CREATE TABLE IF NOT EXISTS bronze.orders_raw (
    ingest_seq NUMBER AUTOINCREMENT START 1 INCREMENT 1 ORDER,
    source STRING,
    raw VARIANT,
    run_id STRING,
    extracted_at TIMESTAMP_TZ,
    src_file STRING,
    src_row NUMBER,
    loaded_at TIMESTAMP_LTZ DEFAULT CURRENT_TIMESTAMP()
)
""")

print("Bronze table is ready!")
```

```
Bronze table is ready!
```

```
Landing file uploaded to Snowflake!
```

```
cur.execute(f"""
COPY INTO bronze.orders_raw
(source, raw, run_id, extracted_at, src_file, src_row)
FROM (
    SELECT
        $1:source::STRING,
        $1:data,
        $1:run_id::STRING,
        $1:extracted_at::TIMESTAMP_TZ,
        METADATA$FILENAME,
        METADATA$FILE_ROW_NUMBER
    FROM @bronze.landing/{stage_dir}/
)
FILE_FORMAT = (FORMAT_NAME = 'bronze.ff_ndjson')
ON_ERROR = 'ABORT_STATEMENT'
""")

print(cur.fetchall())
```

```
[('landing/orders_stream/2026-09-04/bonus_20261113_014351_b93f24/web__bonus_20261113_014351_b93f24.json.gz', 'LOADED', 3, 3, 1, 0, None, None, None, None)]
```

```
cur.execute("SHOW TABLES IN SCHEMA silver")
for row in cur.fetchall():
    print(row[1])
```

```
cur.execute("""
SELECT source, raw
FROM bronze.orders_raw
WHERE raw:order_id::STRING IN (
    'ORD-2001', 'ORD-2002', 'ORD-2003'
)
""")

for row in cur.fetchall():
    print(row)
```

```
: "priya.r@gmail.com",\n  "cust_name": "Priya R",\n  "order_date": "2026-09-03",\n  "order_id": "ORD-2001",\n  "pincode": "600041",\n  "updated_at": "2026-09-03 17:00:00"\n}')
: "arjun.k@yahoo.in",\n  "cust_name": "Arjun K",\n  "order_date": "2026-09-03",\n  "order_id": "ORD-2002",\n  "pincode": "560034",\n  "updated_at": "2026-09-03 17:10:00"\n}')
: "meera@outlook.com",\n  "cust_name": "Meera S",\n  "order_date": "2026-09-03",\n  "order_id": "ORD-2003",\n  "pincode": "400050",\n  "updated_at": "2026-09-03 17:20:00"\n}')
```

```
WHERE order_id IN ('ORD-2001', 'ORD-2002', 'ORD-2003')
""")

for row in cur.fetchall():
    print(row)
```

```
('ORD-2002', 'arjun.k@yahoo.in', Decimal('750.00'))
('ORD-2001', 'priya.r@gmail.com', Decimal('500.00'))
('ORD-2003', 'meera@outlook.com', Decimal('900.00'))
```

```
cur.execute("""
UPDATE silver.orders_clean
SET email = CASE order_id
    WHEN 'ORD-2001' THEN 'priya.r@gmail.com'
    WHEN 'ORD-2002' THEN 'arjun.k@yahoo.in'
    WHEN 'ORD-2003' THEN 'meera@outlook.com'
END
WHERE order_id IN ('ORD-2001', 'ORD-2002', 'ORD-2003')
""")

print("Email addresses corrected!")
```

```
Email addresses corrected!
```

```
SELECT order_id, cust_id, tier, amount_inr, loyalty_points
FROM silver.orders_enriched
WHERE order_id IN ('ORD-2001','ORD-2002','ORD-2003')
""")
print(cur.fetchall())
```

```
[('ORD-2001', 'CUST-01', 'Gold', Decimal('500.00'), 10), ('ORD-2002', 'CUST-02', 'Gold', Decimal('750.00'), 14), ('ORD-2003', 'CUST-03', 'Platinum', Decimal('900.00'), 27)]
```

[65]

```
cur.execute("""
SELECT COUNT(*), SUM(amount_inr), SUM(loyalty_points)
FROM gold.fct_orders
""")
gold_before_replay = cur.fetchone()
print("Gold totals:", gold_before_replay)
```

```
Gold totals: (7, Decimal('9935.00'), 212)
```

```
cur.execute("""
SELECT COUNT(*), SUM(amount_inr), SUM(loyalty_points)
FROM gold.fct_orders
""")

gold_after_replay = cur.fetchone()
print("Before replay:", gold_before_replay)
print("After replay:", gold_after_replay)
print("Idempotency:", gold_before_replay == gold_after_replay)
```

```
Before replay: (7, Decimal('9935.00'), 212)
After replay: (7, Decimal('9935.00'), 212)
Idempotency: True
```